SQL LAB EXAM · OUTPUTS

Every Query with Its Output

All the queries taught in Videos 2–9, run on real sample data. Each query is shown the way you type it at the **mysql>** prompt, followed by exactly what the screen prints.

0 · Sample data used for every output

Video 2 · SELECT, GROUP BY, HAVING & Subqueries

Video 3 · JOINs

Video 4 · Transactions + GRANT / REVOKE

Video 5 · Views & Indexes

Video 6 · Stored Procedures & Functions

Video 7 · Triggers (INSERT / UPDATE / DELETE)

Video 8 · XML + XSD validation

Video 9 · The unseen Library question (from Part 9)

0 · Sample data used for every output

These are the rows inside each table before the queries run. Look back here to see **why** a query gives its answer. Your exam data will be different, so the numbers will differ, but the **shape** of the output is the same. Outputs were produced on MariaDB, which behaves like MySQL; the only difference you might see is an error number (a CHECK failure is 3819 in MySQL 8 and 4025 in MariaDB).

Database: college

```
mysql> USE college;
```

```
Database changed
```

```
mysql> SELECT * FROM Student;
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       101 | Arun    | CSE  | Chennai | 8.50 |
|       102 | Priya   | ECE  | Madurai | 9.10 |
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
3 rows in set
```

```
mysql> SELECT * FROM Course;
```

```
+----------+------------+---------+
| CourseID | CourseName | Credits |
+----------+------------+---------+
| C1       | DBMS       |       4 |
| C2       | Networks   |       3 |
+----------+------------+---------+
2 rows in set
```

```
mysql> SELECT * FROM Enrollment;
```

```
+----------+-----------+----------+-------+
| EnrollID | StudentID | CourseID | Marks |
+----------+-----------+----------+-------+
|        1 |       101 | C1       |    85 |
|        2 |       102 | C2       |    78 |
+----------+-----------+----------+-------+
2 rows in set
```

Database: company

```
mysql> USE company;
```

```
Database changed
```

```
mysql> SELECT * FROM Department;
```

```
+--------+----------+-----------+
| DeptID | DeptName | ManagerID |
+--------+----------+-----------+
|      1 | HR       |       201 |
|      2 | IT       |       202 |
|      3 | Sales    |       204 |
+--------+----------+-----------+
3 rows in set
```

```
mysql> SELECT * FROM Employee;
```

```
+-------+---------+--------+----------+------------+
| EmpID | EmpName | DeptID | Salary   | HireDate   |
+-------+---------+--------+----------+------------+
|   201 | Ravi    |      1 | 45000.00 | 2020-06-15 |
|   202 | Meena   |      2 | 75000.00 | 2019-03-10 |
|   203 | Suresh  |      2 | 52000.00 | 2021-01-05 |
|   204 | Anil    |      3 | 35000.00 | 2023-07-01 |
|   205 | Kavya   |      2 | 61000.00 | 2022-04-12 |
+-------+---------+--------+----------+------------+
5 rows in set
```

Database: railway

```
mysql> USE railway;
```

```
Database changed
```

```
mysql> SELECT * FROM Passenger;
```

```
+-------------+-------+-----+
| PassengerID | Name  | Age |
+-------------+-------+-----+
|           1 | Arun  |  25 |
|           2 | Priya |  30 |
|           3 | Kumar |  40 |
|           4 | Divya |  22 |
+-------------+-------+-----+
4 rows in set
```

```
mysql> SELECT * FROM Train;
```

```
+---------+----------------+---------+-------------+
| TrainID | TrainName      | Source  | Destination |
+---------+----------------+---------+-------------+
|       1 | Vaigai Express | Chennai | Madurai     |
|       2 | Kovai Express  | Chennai | Coimbatore  |
+---------+----------------+---------+-------------+
2 rows in set
```

```
mysql> SELECT * FROM Booking;
```

```
+-----------+-------------+---------+-------------+--------+
| BookingID | PassengerID | TrainID | SeatsBooked | Fare   |
+-----------+-------------+---------+-------------+--------+
|         1 |           1 |       1 |           2 | 500.00 |
|         2 |           2 |       1 |           1 | 300.00 |
|         3 |           1 |       2 |           3 | 900.00 |
|         4 |           4 |       2 |           1 | 450.00 |
+-----------+-------------+---------+-------------+--------+
4 rows in set
```

Database: hospital

```
mysql> USE hospital;
```

```
Database changed
```

```
mysql> SELECT * FROM Patient;
```

```
+-----------+-------------+---------+
| PatientID | PatientName | City    |
+-----------+-------------+---------+
|         1 | Arun        | Chennai |
|         2 | Priya       | Madurai |
|         3 | Kavya       | Salem   |
+-----------+-------------+---------+
3 rows in set
```

```
mysql> SELECT * FROM Doctor;
```

```
+----------+------------+----------------+
| DoctorID | DoctorName | Specialization |
+----------+------------+----------------+
|        1 | Dr. Ravi   | Cardiology     |
|        2 | Dr. Meena  | Neurology      |
|        3 | Dr. Kumar  | Ortho          |
+----------+------------+----------------+
3 rows in set
```

```
mysql> SELECT * FROM Appointment;
```

```
+-------+-----------+----------+------------+--------+
| AppID | PatientID | DoctorID | VisitDate  | Fee    |
+-------+-----------+----------+------------+--------+
|     1 |         1 |        1 | 2025-01-10 | 500.00 |
|     2 |         2 |        1 | 2025-01-11 | 500.00 |
|     3 |         1 |        2 | 2025-01-12 | 800.00 |
|     4 |         3 |        1 | 2025-01-15 | 600.00 |
+-------+-----------+----------+------------+--------+
4 rows in set
```

Database: bank

```
mysql> USE bank;
```

```
Database changed
```

```
mysql> SELECT * FROM Customer;
```

```
+------------+--------------+---------+
| CustomerID | CustomerName | City    |
+------------+--------------+---------+
|          1 | Arun         | Chennai |
|          2 | Priya        | Madurai |
|          3 | Kumar        | Salem   |
|          4 | Divya        | Trichy  |
+------------+--------------+---------+
4 rows in set
```

```
mysql> SELECT * FROM Account;
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1001 |          1 | Savings     | 50000.00 |
|      1002 |          2 | Current     | 80000.00 |
|      1003 |          3 | Savings     | 20000.00 |
|      1004 |          4 | Savings     | 15000.00 |
+-----------+------------+-------------+----------+
4 rows in set
```

```
mysql> SELECT * FROM `Transaction`;
```

```
+-------+-----------+---------+---------+
| TxnID | AccountNo | Amount  | TxnType |
+-------+-----------+---------+---------+
|     1 |      1001 | 5000.00 | Credit  |
+-------+-----------+---------+---------+
1 row in set
```

Database: airline

```
mysql> USE airline;
```

```
Database changed
```

```
mysql> SELECT * FROM Passenger;
```

```
+-------------+-------+
| PassengerID | Name  |
+-------------+-------+
|           1 | Arun  |
|           2 | Priya |
|           3 | Kumar |
+-------------+-------+
3 rows in set
```

```
mysql> SELECT * FROM Flight;
```

```
+----------+--------------+---------+-------------+-------------+
| FlightID | FlightName   | Source  | Destination | TicketPrice |
+----------+--------------+---------+-------------+-------------+
| F1       | IndiGo 6E201 | Chennai | Delhi       |     6000.00 |
| F2       | Air India 440| Chennai | Mumbai      |     4200.00 |
| F3       | SpiceJet 112 | Chennai | Bengaluru   |     3500.00 |
+----------+--------------+---------+-------------+-------------+
3 rows in set
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 12A    |
|        2 |           1 | F2       | 5B     |
+----------+-------------+----------+--------+
2 rows in set
```

Database: cinema

```
mysql> USE cinema;
```

```
Database changed
```

```
mysql> SELECT * FROM Movie;
```

```
+---------+-----------+----------+-------------+
| MovieID | MovieName | Genre    | TicketPrice |
+---------+-----------+----------+-------------+
|       1 | Leo       | Action   |      200.00 |
|       2 | Jailer    | Action   |      150.00 |
|       3 | Vikram    | Thriller |      150.00 |
|       4 | Amaran    | Drama    |      180.00 |
+---------+-----------+----------+-------------+
4 rows in set
```

```
mysql> SELECT * FROM Customer;
```

```
+------------+--------------+
| CustomerID | CustomerName |
+------------+--------------+
|          1 | Arun         |
|          2 | Priya        |
|          3 | Kumar        |
+------------+--------------+
3 rows in set
```

```
mysql> SELECT * FROM Booking;
```

```
+-----------+------------+---------+-------------+
| BookingID | CustomerID | MovieID | SeatsBooked |
+-----------+------------+---------+-------------+
|         1 |          1 |       1 |           4 |
|         2 |          2 |       1 |           5 |
|         3 |          1 |       2 |           2 |
|         4 |          3 |       3 |           3 |
+-----------+------------+---------+-------------+
4 rows in set
```

Database: food

```
mysql> USE food;
```

```
Database changed
```

```
mysql> SELECT * FROM Restaurant;
```

```
+--------------+------------------+---------+
| RestaurantID | RestaurantName   | City    |
+--------------+------------------+---------+
|            1 | A2B              | Chennai |
|            2 | Murugan Idli     | Madurai |
|            3 | Dindigul Biryani | Chennai |
|            4 | Adyar Bakery     | Chennai |
+--------------+------------------+---------+
4 rows in set
```

```
mysql> SELECT * FROM Customer;
```

```
+------------+--------------+---------+
| CustomerID | CustomerName | City    |
+------------+--------------+---------+
|          1 | Arun         | Chennai |
|          2 | Priya        | Madurai |
+------------+--------------+---------+
2 rows in set
```

```
mysql> SELECT * FROM FoodOrder;
```

```
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       1 |          1 |            1 |        2 | 300.00 |    4.5 |
|       2 |          2 |            2 |        3 | 150.00 |    5.0 |
|       3 |          1 |            1 |        1 | 200.00 |    3.9 |
|       4 |          2 |            3 |        2 | 400.00 |    4.8 |
+---------+------------+--------------+----------+--------+--------+
4 rows in set
```

Video 2 · SELECT, GROUP BY, HAVING & Subqueries

WHERE: exact match

database: college

```
mysql> SELECT * FROM Student WHERE City = 'Chennai';
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       101 | Arun    | CSE  | Chennai | 8.50 |
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
2 rows in set
```

WHERE with AND

database: college

```
mysql> SELECT * FROM Student WHERE Dept = 'CSE' AND CGPA > 8;
```

```
+-----------+------+------+---------+------+
| StudentID | Name | Dept | City    | CGPA |
+-----------+------+------+---------+------+
|       101 | Arun | CSE  | Chennai | 8.50 |
+-----------+------+------+---------+------+
1 row in set
```

WHERE with BETWEEN

database: company

```
mysql> SELECT * FROM Employee WHERE Salary BETWEEN 30000 AND
60000;
```

```
+-------+---------+--------+----------+------------+
| EmpID | EmpName | DeptID | Salary   | HireDate   |
+-------+---------+--------+----------+------------+
|   201 | Ravi    |      1 | 45000.00 | 2020-06-15 |
|   203 | Suresh  |      2 | 52000.00 | 2021-01-05 |
|   204 | Anil    |      3 | 35000.00 | 2023-07-01 |
+-------+---------+--------+----------+------------+
3 rows in set
```

BETWEEN includes both ends.

WHERE with IN (list)

database: college

```
mysql> SELECT * FROM Student WHERE City IN ('Chennai',
'Madurai');
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       101 | Arun    | CSE  | Chennai | 8.50 |
|       102 | Priya   | ECE  | Madurai | 9.10 |
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
3 rows in set
```

WHERE with LIKE

database: college

```
mysql> SELECT * FROM Student WHERE Name LIKE 'A%';
```

```
+-----------+------+------+---------+------+
| StudentID | Name | Dept | City    | CGPA |
+-----------+------+------+---------+------+
|       101 | Arun | CSE  | Chennai | 8.50 |
+-----------+------+------+---------+------+
1 row in set
```

'A%' = starts with A.

WHERE with IS NULL

database: hospital

```
mysql> SELECT * FROM Doctor WHERE Specialization IS NULL;
```

```
Empty set
```

No doctor has a NULL specialization (it is NOT NULL), so the answer is an empty set.

ORDER BY ... DESC

database: airline

```
mysql> SELECT * FROM Flight ORDER BY TicketPrice DESC;
```

```
+----------+--------------+---------+-------------+-------------+
| FlightID | FlightName   | Source  | Destination | TicketPrice |
+----------+--------------+---------+-------------+-------------+
| F1       | IndiGo 6E201 | Chennai | Delhi       |     6000.00 |
| F2       | Air India 440| Chennai | Mumbai      |     4200.00 |
| F3       | SpiceJet 112 | Chennai | Bengaluru   |     3500.00 |
+----------+--------------+---------+-------------+-------------+
3 rows in set
```

> Highest price first.

ORDER BY (ASC is default)

database: college

```
mysql> SELECT * FROM Student ORDER BY Name;
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       101 | Arun    | CSE  | Chennai | 8.50 |
|       103 | Karthik | CSE  | Chennai | 7.20 |
|       102 | Priya   | ECE  | Madurai | 9.10 |
+-----------+---------+------+---------+------+
3 rows in set
```

COUNT

database: company

```
mysql> SELECT COUNT(*) FROM Employee;
```

```
+----------+
| COUNT(*) |
+----------+
|        5 |
+----------+
1 row in set
```

SUM

database: railway

```
mysql> SELECT SUM(Fare) FROM Booking;
```

```
+-----------+
| SUM(Fare) |
+-----------+
|   2150.00 |
+-----------+
1 row in set
```

AVG with WHERE

database: college

```
mysql> SELECT AVG(CGPA) FROM Student WHERE Dept = 'CSE';
```

```
+-----------+
| AVG(CGPA) |
+-----------+
|  7.850000 |
+-----------+
1 row in set
```

> (8.50 + 7.20) / 2 = 7.85

MAX and MIN

database: company

```
mysql> SELECT MAX(Salary), MIN(Salary) FROM Employee;
```

```
+-------------+-------------+
| MAX(Salary) | MIN(Salary) |
+-------------+-------------+
|    75000.00 |    35000.00 |
+-------------+-------------+
1 row in set
```

Alias with AS

database: college

```
mysql> SELECT AVG(CGPA) AS Avg_CSE_CGPA FROM Student WHERE
Dept = 'CSE';
```

```
+--------------+
| Avg_CSE_CGPA |
+--------------+
|     7.850000 |
+--------------+
1 row in set
```

> Same answer, nicer column name.

GROUP BY: highest salary in each department

database: company

```
mysql> SELECT DeptID, MAX(Salary) AS HighestSalary
    -> FROM Employee
    -> GROUP BY DeptID;
```

```
+--------+---------------+
| DeptID | HighestSalary |
+--------+---------------+
|      1 |      45000.00 |
|      2 |      75000.00 |
|      3 |      35000.00 |
+--------+---------------+
3 rows in set
```

GROUP BY: total fare collected for each train

database: railway

```
mysql> SELECT TrainID, SUM(Fare) AS TotalFare
    -> FROM Booking
    -> GROUP BY TrainID;
```

```
+---------+-----------+
| TrainID | TotalFare |
+---------+-----------+
|       1 |    800.00 |
|       2 |   1350.00 |
+---------+-----------+
2 rows in set
```

HAVING: departments with more than one employee

database: company

```
mysql> SELECT DeptID, COUNT(*) AS EmpCount
    -> FROM Employee
    -> GROUP BY DeptID
    -> HAVING COUNT(*) > 1;
```

```
+--------+----------+
| DeptID | EmpCount |
+--------+----------+
|      2 |        3 |
+--------+----------+
1 row in set
```

Only Dept 2 (IT) has more than one employee (Meena, Suresh, Kavya).

HAVING: movies with average seats > 3

database: cinema

```
mysql> SELECT MovieID, AVG(SeatsBooked) AS AvgSeats
    -> FROM Booking
    -> GROUP BY MovieID
    -> HAVING AVG(SeatsBooked) > 3;
```

```
+---------+----------+
| MovieID | AvgSeats |
+---------+----------+
|       1 |   4.5000 |
+---------+----------+
1 row in set
```

Movie 1: (4 + 5) / 2 = 4.5. Movies 2 and 3 have averages 2 and 3, so they are filtered out.

WHERE + GROUP BY: per city, rating ≥ 4.2

database: food

```
mysql> SELECT r.City,
    ->        COUNT(f.OrderID) AS TotalOrders,
    ->        AVG(f.Price)     AS AvgPrice,
    ->        SUM(f.Quantity)  AS TotalQty
    -> FROM FoodOrder f
```

```
    -> JOIN Restaurant r ON f.RestaurantID = r.RestaurantID
    -> WHERE f.Rating >= 4.2
    -> GROUP BY r.City;
```

```
+---------+-------------+------------+----------+
| City    | TotalOrders | AvgPrice   | TotalQty |
+---------+-------------+------------+----------+
| Chennai |           2 | 350.000000 |        4 |
| Madurai |           1 | 150.000000 |        3 |
+---------+-------------+------------+----------+
2 rows in set
```

Order 3 (rating 3.9) is removed by WHERE before grouping.

Count appointments handled by each doctor

database: hospital

```
mysql> SELECT DoctorID, COUNT(*) AS TotalAppointments
    -> FROM Appointment
    -> GROUP BY DoctorID;
```

```
+----------+------------------+
| DoctorID | TotalAppointments |
+----------+------------------+
|        1 |                3 |
|        2 |                1 |
+----------+------------------+
2 rows in set
```

Subquery: students with CGPA > average

database: college

```
mysql> SELECT Name FROM Student
    -> WHERE CGPA > (SELECT AVG(CGPA) FROM Student);
```

```
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
+-------+
2 rows in set
```

Average = (8.50 + 9.10 + 7.20) / 3 = 8.27.

Subquery: employees earning more than average

database: company

```
mysql> SELECT EmpID, EmpName, Salary FROM Employee
    -> WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

```
+-------+---------+----------+
| EmpID | EmpName | Salary   |
+-------+---------+----------+
|   202 | Meena   | 75000.00 |
|   205 | Kavya   | 61000.00 |
+-------+---------+----------+
2 rows in set
```

Average salary = 53600.

Subquery: accounts with balance > average

database: bank

```
mysql> SELECT AccountNo, CustomerID, Balance FROM Account
    -> WHERE Balance > (SELECT AVG(Balance) FROM Account);
```

```
+-----------+------------+----------+
| AccountNo | CustomerID | Balance  |
+-----------+------------+----------+
|      1001 |          1 | 50000.00 |
|      1002 |          2 | 80000.00 |
+-----------+------------+----------+
2 rows in set
```

Average balance = 41250.

Subquery: account with the maximum balance

database: bank

```
mysql> SELECT * FROM Account
    -> WHERE Balance = (SELECT MAX(Balance) FROM Account);
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1002 |          2 | Current     | 80000.00 |
+-----------+------------+-------------+----------+
1 row in set
```

IN: passengers who made at least one booking

database: railway

```
mysql> SELECT Name FROM Passenger
    -> WHERE PassengerID IN (SELECT PassengerID FROM
Booking);
```

```
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
| Divya |
+-------+
3 rows in set
```

> Kumar has no booking, so he is not listed.

IN: doctors who have at least one appointment

database: hospital

```
mysql> SELECT * FROM Doctor
    -> WHERE DoctorID IN (SELECT DoctorID FROM Appointment);
```

```
+----------+------------+----------------+
| DoctorID | DoctorName | Specialization |
+----------+------------+----------------+
|        1 | Dr. Ravi   | Cardiology     |
|        2 | Dr. Meena  | Neurology      |
+----------+------------+----------------+
2 rows in set
```

> Dr. Kumar has no appointment.

IN with WHERE inside: restaurants with an order rated 5

database: food

```
mysql> SELECT * FROM Restaurant
    -> WHERE RestaurantID IN (SELECT RestaurantID FROM
FoodOrder WHERE Rating = 5);
```

```
+--------------+----------------+---------+
| RestaurantID | RestaurantName | City    |
+--------------+----------------+---------+
|            2 | Murugan Idli   | Madurai |
+--------------+----------------+---------+
1 row in set
```

NOT IN: restaurants that never received an order

database: food

```
mysql> SELECT * FROM Restaurant
    -> WHERE RestaurantID NOT IN (SELECT RestaurantID FROM
FoodOrder);
```

```
+--------------+----------------+---------+
| RestaurantID | RestaurantName | City    |
+--------------+----------------+---------+
|            4 | Adyar Bakery   | Chennai |
+--------------+----------------+---------+
1 row in set
```

Correlated: fare > average fare of that same train

database: railway

```
mysql> SELECT * FROM Booking b1
    -> WHERE Fare > (SELECT AVG(Fare)
    ->               FROM Booking b2
    ->               WHERE b2.TrainID = b1.TrainID);
```

```
+-----------+-------------+---------+-------------+--------+
| BookingID | PassengerID | TrainID | SeatsBooked | Fare   |
+-----------+-------------+---------+-------------+--------+
|         1 |           1 |       1 |           2 | 500.00 |
|         3 |           1 |       2 |           3 | 900.00 |
+-----------+-------------+---------+-------------+--------+
2 rows in set
```

> Train 1 average = 400 → booking 1 (500). Train 2 average = 675 → booking 3 (900).

Video 3 · JOINs

INNER JOIN (2 tables)

database: company

```
mysql> SELECT e.EmpName, e.Salary, d.DeptName
    -> FROM Employee e
    -> JOIN Department d ON e.DeptID = d.DeptID;
```

```
+---------+----------+----------+
| EmpName | Salary   | DeptName |
+---------+----------+----------+
| Ravi    | 45000.00 | HR       |
| Meena   | 75000.00 | IT       |
| Suresh  | 52000.00 | IT       |
| Anil    | 35000.00 | Sales    |
| Kavya   | 61000.00 | IT       |
+---------+----------+----------+
5 rows in set
```

3-table JOIN: patient and doctor names

database: hospital

```
mysql> SELECT p.PatientName, d.DoctorName
    -> FROM Appointment a
    -> JOIN Patient p ON a.PatientID = p.PatientID
    -> JOIN Doctor  d ON a.DoctorID  = d.DoctorID;
```

```
+-------------+------------+
| PatientName | DoctorName |
+-------------+------------+
| Arun        | Dr. Ravi   |
| Priya       | Dr. Ravi   |
| Arun        | Dr. Meena  |
| Kavya       | Dr. Ravi   |
+-------------+------------+
4 rows in set
```

3-table JOIN: student and course names

database: college

```
mysql> SELECT s.Name, c.CourseName
    -> FROM Student s
```

```
    -> JOIN Enrollment e ON s.StudentID = e.StudentID
    -> JOIN Course     c ON e.CourseID  = c.CourseID;
```

```
+-------+------------+
| Name  | CourseName |
+-------+------------+
| Arun  | DBMS       |
| Priya | Networks   |
+-------+------------+
2 rows in set
```

> Karthik is not enrolled, so a normal JOIN leaves him out.

LEFT JOIN: include passengers with no booking

database: railway

```
mysql> SELECT p.PassengerID, p.Name, b.BookingID, b.TrainID,
b.Fare
    -> FROM Passenger p
    -> LEFT JOIN Booking b ON p.PassengerID = b.PassengerID;
```

```
+-------------+-------+-----------+---------+--------+
| PassengerID | Name  | BookingID | TrainID | Fare   |
+-------------+-------+-----------+---------+--------+
|           1 | Arun  |         1 |       1 | 500.00 |
|           2 | Priya |         2 |       1 | 300.00 |
|           1 | Arun  |         3 |       2 | 900.00 |
|           4 | Divya |         4 |       2 | 450.00 |
|           3 | Kumar | NULL      | NULL    | NULL   |
+-------------+-------+-----------+---------+--------+
5 rows in set
```

> Kumar appears with NULLs because he has no booking.

FULL OUTER JOIN (LEFT UNION RIGHT)

database: cinema

```
mysql> SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
    -> FROM Booking b LEFT JOIN Movie m ON b.MovieID =
m.MovieID
    -> UNION
    -> SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
```

```
    -> FROM Booking b RIGHT JOIN Movie m ON b.MovieID =
m.MovieID;
```

```
+-----------+-------------+---------+-----------+
| BookingID | SeatsBooked | MovieID | MovieName |
+-----------+-------------+---------+-----------+
|         1 |           4 |       1 | Leo       |
|         2 |           5 |       1 | Leo       |
|         3 |           2 |       2 | Jailer    |
|         4 |           3 |       3 | Vikram    |
| NULL      | NULL        |       4 | Amaran    |
+-----------+-------------+---------+-----------+
5 rows in set
```

> Amaran has no booking, so its booking columns are NULL (the 'blanks').

CROSS JOIN: all passenger–flight combinations

database: airline

```
mysql> SELECT p.Name, f.FlightName
    -> FROM Passenger p
    -> CROSS JOIN Flight f;
```

```
+-------+---------------+
| Name  | FlightName    |
+-------+---------------+
| Arun  | IndiGo 6E201  |
| Priya | IndiGo 6E201  |
| Kumar | IndiGo 6E201  |
| Arun  | Air India 440 |
| Priya | Air India 440 |
| Kumar | Air India 440 |
| Arun  | SpiceJet 112  |
| Priya | SpiceJet 112  |
| Kumar | SpiceJet 112  |
+-------+---------------+
9 rows in set
```

> 3 passengers × 3 flights = 9 rows.

SELF JOIN: pairs of employees in the same department

database: company

```
mysql> SELECT e1.EmpName AS Employee1, e2.EmpName AS
Employee2, e1.DeptID
```

```
    -> FROM Employee e1
    -> JOIN Employee e2
    ->   ON e1.DeptID = e2.DeptID
    ->  AND e1.EmpID < e2.EmpID;
```

```
+-----------+-----------+--------+
| Employee1 | Employee2 | DeptID |
+-----------+-----------+--------+
| Meena     | Suresh    |      2 |
| Meena     | Kavya     |      2 |
| Suresh    | Kavya     |      2 |
+-----------+-----------+--------+
3 rows in set
```

> Only Dept 2 has more than one person, so all pairs come from it.

Video 4 · Transactions + GRANT / REVOKE

Each block starts from the original sample data. Watch the SELECT outputs: they prove what COMMIT keeps and what ROLLBACK undoes.

COMMIT: insert student + enrollment

database: college

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Student VALUES (104, 'Divya', 'IT',
'Coimbatore', 8.80);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Enrollment VALUES (3, 104, 'C1', 90);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Enrollment;
```

```
+----------+-----------+----------+-------+
| EnrollID | StudentID | CourseID | Marks |
+----------+-----------+----------+-------+
|        1 |       101 | C1       |    85 |
|        2 |       102 | C2       |    78 |
|        3 |       104 | C1       |    90 |
+----------+-----------+----------+-------+
3 rows in set
```

After COMMIT the new rows are permanent.

ROLLBACK: delete then restore

database: college

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Enrollment WHERE EnrollID = 2;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Enrollment;
```

```
+----------+-----------+----------+-------+
| EnrollID | StudentID | CourseID | Marks |
+----------+-----------+----------+-------+
|        1 |       101 | C1       |    85 |
|        3 |       104 | C1       |    90 |
+----------+-----------+----------+-------+
2 rows in set
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Enrollment;
```

```
+----------+-----------+----------+-------+
| EnrollID | StudentID | CourseID | Marks |
+----------+-----------+----------+-------+
|        1 |       101 | C1       |    85 |
|        2 |       102 | C2       |    78 |
|        3 |       104 | C1       |    90 |
+----------+-----------+----------+-------+
3 rows in set
```

> Row 2 disappears, then comes back after ROLLBACK.

ROLLBACK undoes BOTH changes

database: college

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Student SET CGPA = 9.50 WHERE StudentID = 103;
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Enrollment VALUES (4, 103, 'C2', 70);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Student WHERE StudentID = 103;
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
1 row in set
```

```
mysql> SELECT * FROM Enrollment WHERE EnrollID = 4;
```

```
Empty set
```

> CGPA is still 7.20 and enrollment 4 does not exist.

SAVEPOINT + ROLLBACK TO

database: college

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Student VALUES (105, 'Rahul', 'MECH',
'Salem', 7.90);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Enrollment VALUES (5, 105, 'C1', 65);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Student    WHERE StudentID = 105;
```

```
+-----------+-------+------+-------+------+
| StudentID | Name  | Dept | City  | CGPA |
+-----------+-------+------+-------+------+
|       105 | Rahul | MECH | Salem | 7.90 |
+-----------+-------+------+-------+------+
1 row in set
```

```
mysql> SELECT * FROM Enrollment WHERE StudentID = 105;
```

```
Empty set
```

> The student (before the savepoint) stays; the enrollment (after it) is undone.

+10%, savepoint, +5%, roll back to the savepoint

database: company

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT EmpID, EmpName, Salary FROM Employee;
```

```
+-------+---------+----------+
| EmpID | EmpName | Salary   |
+-------+---------+----------+
|   201 | Ravi    | 45000.00 |
|   202 | Meena   | 75000.00 |
|   203 | Suresh  | 52000.00 |
|   204 | Anil    | 35000.00 |
|   205 | Kavya   | 61000.00 |
+-------+---------+----------+
5 rows in set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET Salary = Salary * 1.10;
```

```
Query OK, 5 rows affected
```

```
mysql> SAVEPOINT after_10;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET Salary = Salary * 1.05;
```

```
Query OK, 5 rows affected
```

```
mysql> SELECT EmpID, EmpName, Salary FROM Employee;
```

```
+-------+---------+----------+
| EmpID | EmpName | Salary   |
+-------+---------+----------+
|   201 | Ravi    | 51975.00 |
|   202 | Meena   | 86625.00 |
|   203 | Suresh  | 60060.00 |
|   204 | Anil    | 40425.00 |
|   205 | Kavya   | 70455.00 |
+-------+---------+----------+
5 rows in set
```

```
mysql> ROLLBACK TO after_10;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT EmpID, EmpName, Salary FROM Employee;
```

```
+-------+---------+----------+
| EmpID | EmpName | Salary   |
+-------+---------+----------+
|   201 | Ravi    | 49500.00 |
|   202 | Meena   | 82500.00 |
|   203 | Suresh  | 57200.00 |
|   204 | Anil    | 38500.00 |
|   205 | Kavya   | 67100.00 |
+-------+---------+----------+
5 rows in set
```

> Final salaries = original × 1.10 (the extra 5% was rolled back).

Book two tickets, roll back only the second

database: airline

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Ticket VALUES (3, 1, 'F1', '1A');
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT after_first;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Ticket VALUES (4, 2, 'F2', '2B');
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO after_first;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 12A    |
|        2 |           1 | F2       | 5B     |
|        3 |           1 | F1       | 1A     |
+----------+-------------+----------+--------+
3 rows in set
```

> Ticket 3 is saved, ticket 4 is gone.

Bank transfer (all-or-nothing)

database: bank

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Account SET Balance = Balance - 10000 WHERE
AccountNo = 1002;
```

```
Query OK, 1 row affected
```

```
mysql> UPDATE Account SET Balance = Balance + 10000 WHERE
AccountNo = 1003;
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO `Transaction` VALUES (3, 1002, 10000,
'Debit');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO `Transaction` VALUES (4, 1003, 10000,
'Credit');
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Account WHERE AccountNo IN (1002, 1003);
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1002 |          2 | Current     | 70000.00 |
|      1003 |          3 | Savings     | 30000.00 |
+-----------+------------+-------------+----------+
2 rows in set
```

```
mysql> SELECT * FROM `Transaction`;
```

```
+-------+-----------+----------+---------+
| TxnID | AccountNo | Amount   | TxnType |
+-------+-----------+----------+---------+
|     1 |      1001 |  5000.00 | Credit  |
|     3 |      1002 | 10000.00 | Debit   |
|     4 |      1003 | 10000.00 | Credit  |
+-------+-----------+----------+---------+
3 rows in set
```

> 1002: 80000 → 70000, 1003: 20000 → 30000, both transactions recorded.

What a FAILED transfer looks like → ROLLBACK

database: bank

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Account SET Balance = Balance - 10000 WHERE
AccountNo = 1004;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Account WHERE AccountNo = 1004;
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1004 |          4 | Savings     | 15000.00 |
+-----------+------------+-------------+----------+
1 row in set
```

Suppose a later step of the transfer failed after this UPDATE. Typing ROLLBACK undoes the deduction, so the balance is back to 15000. Nothing is half-done.

GRANT, REVOKE and SHOW GRANTS

database: college

```
mysql> CREATE USER 'student_user'@'localhost' IDENTIFIED BY
'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER 'faculty_user'@'localhost' IDENTIFIED BY
'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER 'course_admin'@'localhost' IDENTIFIED BY
'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON college.Student TO
'student_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON college.Student TO
'faculty_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON college.Enrollment TO
'faculty_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> REVOKE INSERT ON college.Enrollment FROM
'faculty_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON college.Course TO
'course_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'faculty_user'@'localhost';
```

```
+-----------------------------------------------------------------------------+
| Grants for faculty_user@localhost                                           |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `faculty_user`@`localhost`                            |
| GRANT SELECT, INSERT, UPDATE ON `college`.`Student` TO `faculty_user`@`localhost` |
| GRANT SELECT ON `college`.`Enrollment` TO `faculty_user`@`localhost`        |
+-----------------------------------------------------------------------------+
3 rows in set
```

```
mysql> SHOW GRANTS FOR 'course_admin'@'localhost';
```

```
+-----------------------------------------------------------------------------+
| Grants for course_admin@localhost                                           |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `course_admin`@`localhost`                            |
| GRANT ALL PRIVILEGES ON `college`.`Course` TO `course_admin`@`localhost` |
+-----------------------------------------------------------------------------+
2 rows in set
```

> After the REVOKE, faculty_user keeps only SELECT on Enrollment. 'USAGE' just means the user can log in.

Video 5 · Views & Indexes

View: Ticket_Details_View (3 tables)

database: airline

```
mysql> CREATE VIEW Ticket_Details_View AS
    -> SELECT t.TicketID, p.Name, f.FlightName, f.Source,
f.Destination, t.SeatNo
    -> FROM Ticket t
    -> JOIN Passenger p ON t.PassengerID = p.PassengerID
    -> JOIN Flight    f ON t.FlightID    = f.FlightID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket_Details_View;
```

```
+----------+-------+---------------+---------+-------------+--------+
| TicketID | Name  | FlightName    | Source  | Destination | SeatNo |
+----------+-------+---------------+---------+-------------+--------+
|        2 | Arun  | Air India 440 | Chennai | Mumbai      | 5B     |
|        1 | Kumar | IndiGo 6E201  | Chennai | Delhi       | 12A    |
+----------+-------+---------------+---------+-------------+--------+
2 rows in set
```

Index on Flight(Source)

database: airline

```
mysql> CREATE INDEX idx_flight_source ON Flight(Source);
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW INDEX FROM Flight;
```

```
+--------+------------+------------------+--------------+-------------+
| Table  | Non_unique | Key_name         | Seq_in_index | Column_name |
+--------+------------+------------------+--------------+-------------+
| Flight |          0 | PRIMARY          |            1 | FlightID    |
| Flight |          1 | idx_flight_source |            1 | Source      |
+--------+------------+------------------+--------------+-------------+
2 rows in set
```

> SHOW INDEX prints many columns; only the important ones are shown here. PRIMARY is created automatically for the primary key.

View: Employee_Department_View

database: company

```
mysql> CREATE VIEW Employee_Department_View AS
    -> SELECT e.EmpID, e.EmpName, e.Salary, d.DeptName
    -> FROM Employee e JOIN Department d ON e.DeptID =
d.DeptID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Employee_Department_View;
```

```
+-------+---------+----------+----------+
| EmpID | EmpName | Salary   | DeptName |
+-------+---------+----------+----------+
|   201 | Ravi    | 45000.00 | HR       |
|   202 | Meena   | 75000.00 | IT       |
|   203 | Suresh  | 52000.00 | IT       |
|   204 | Anil    | 35000.00 | Sales    |
|   205 | Kavya   | 61000.00 | IT       |
+-------+---------+----------+----------+
5 rows in set
```

View: Customer_Account_View

database: bank

```
mysql> CREATE VIEW Customer_Account_View AS
    -> SELECT c.CustomerID, c.CustomerName, a.AccountNo,
a.AccountType, a.Balance
    -> FROM Customer c JOIN Account a ON c.CustomerID =
a.CustomerID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Customer_Account_View;
```

```
+------------+--------------+-----------+-------------+----------+
| CustomerID | CustomerName | AccountNo | AccountType | Balance  |
+------------+--------------+-----------+-------------+----------+
|          1 | Arun         |      1001 | Savings     | 50000.00 |
|          2 | Priya        |      1002 | Current     | 80000.00 |
|          3 | Kumar        |      1003 | Savings     | 20000.00 |
|          4 | Divya        |      1004 | Savings     | 15000.00 |
+------------+--------------+-----------+-------------+----------+
4 rows in set
```

View: Student_Course_View

database: college

```
mysql> CREATE VIEW Student_Course_View AS
    -> SELECT s.StudentID, s.Name, c.CourseName, e.Marks
    -> FROM Student s
    -> JOIN Enrollment e ON s.StudentID = e.StudentID
    -> JOIN Course     c ON e.CourseID  = c.CourseID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Student_Course_View;
```

```
+-----------+-------+------------+-------+
| StudentID | Name  | CourseName | Marks |
+-----------+-------+------------+-------+
|       101 | Arun  | DBMS       |    85 |
|       102 | Priya | Networks   |    78 |
+-----------+-------+------------+-------+
2 rows in set
```

Index on Student(Dept)

database: college

```
mysql> CREATE INDEX idx_student_dept ON Student(Dept);
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW INDEX FROM Student;
```

```
+---------+------------+------------------+--------------+-------------+
| Table   | Non_unique | Key_name         | Seq_in_index | Column_name |
+---------+------------+------------------+--------------+-------------+
| Student |          0 | PRIMARY          |            1 | StudentID   |
| Student |          0 | Name             |            1 | Name        |
| Student |          1 | idx_student_dept |            1 | Dept        |
+---------+------------+------------------+--------------+-------------+
3 rows in set
```

> Name also has an index automatically, because it is UNIQUE.

Video 6 · Stored Procedures & Functions

Table used by the student procedures

```
mysql> CREATE DATABASE plsql;
```

```
Query OK, 0 rows affected
```

```
mysql> USE plsql;
```

```
Database changed
```

```
mysql> CREATE TABLE Student (StudentID INT PRIMARY KEY, Name
VARCHAR(50),
    ->                       Dept VARCHAR(20), Marks INT);
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Student VALUES (1,'Arun','CSE',72),
(2,'Priya','IT',35), (3,'Kumar','ECE',58);
```

```
Query OK, 3 rows affected
```

```
mysql> SELECT * FROM Student;
```

```
+-----------+-------+------+-------+
| StudentID | Name  | Dept | Marks |
+-----------+-------+------+-------+
|         1 | Arun  | CSE  |    72 |
|         2 | Priya | IT   |    35 |
|         3 | Kumar | ECE  |    58 |
+-----------+-------+------+-------+
3 rows in set
```

Procedure: welcome message

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE Welcome()
    -> BEGIN
```

```
    ->     SELECT 'Welcome to the DBMS Lab!' AS Message;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> CALL Welcome();
```

```
+--------------------------+
| Message                  |
+--------------------------+
| Welcome to the DBMS Lab! |
+--------------------------+
1 row in set
```

Procedure: display a student by ID

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE GetStudent(IN p_id INT)
    -> BEGIN
    ->     SELECT * FROM Student WHERE StudentID = p_id;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> CALL GetStudent(2);
```

```
+-----------+-------+------+-------+
| StudentID | Name  | Dept | Marks |
+-----------+-------+------+-------+
|         2 | Priya | IT   |    35 |
+-----------+-------+------+-------+
1 row in set
```

Procedure: increase marks of a specified student

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE IncreaseMarks(IN p_id INT, IN p_inc
INT)
    -> BEGIN
    ->     UPDATE Student SET Marks = Marks + p_inc WHERE
StudentID = p_id;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> CALL IncreaseMarks(1, 5);
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Student;
```

```
+-----------+-------+------+-------+
| StudentID | Name  | Dept | Marks |
+-----------+-------+------+-------+
|         1 | Arun  | CSE  |    77 |
|         2 | Priya | IT   |    35 |
|         3 | Kumar | ECE  |    58 |
+-----------+-------+------+-------+
3 rows in set
```

Arun: 72 → 77.

Procedure: delete a student record

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE PROCEDURE DeleteStudent(IN p_id INT)
    -> BEGIN
    ->     DELETE FROM Student WHERE StudentID = p_id;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> CALL DeleteStudent(3);
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Student;
```

```
+-----------+-------+------+-------+
| StudentID | Name  | Dept | Marks |
+-----------+-------+------+-------+
|         1 | Arun  | CSE  |    77 |
|         2 | Priya | IT   |    35 |
+-----------+-------+------+-------+
2 rows in set
```

> Kumar (ID 3) is gone.

Function: square of a number

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE FUNCTION Square(n INT) RETURNS INT
DETERMINISTIC
    -> BEGIN
    ->     RETURN n * n;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> SELECT Square(7);
```

```
+-----------+
| Square(7) |
+-----------+
|        49 |
+-----------+
1 row in set
```

Function: add two numbers

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE FUNCTION AddNumbers(a INT, b INT) RETURNS INT
DETERMINISTIC
    -> BEGIN
    ->     RETURN a + b;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> SELECT AddNumbers(10, 20);
```

```
+--------------------+
| AddNumbers(10, 20) |
+--------------------+
|                 30 |
+--------------------+
1 row in set
```

Function: pass or fail

database: plsql

```
mysql> DELIMITER //
```

```
mysql> CREATE FUNCTION PassOrFail(m INT) RETURNS VARCHAR(10)
DETERMINISTIC
    -> BEGIN
    ->     IF m >= 40 THEN
    ->         RETURN 'PASS';
    ->     ELSE
    ->         RETURN 'FAIL';
    ->     END IF;
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> SELECT PassOrFail(65);
```

```
+----------------+
| PassOrFail(65) |
+----------------+
| PASS           |
+----------------+
1 row in set
```

```
mysql> SELECT Name, Marks, PassOrFail(Marks) AS Result FROM
Student;
```

```
+-------+-------+--------+
| Name  | Marks | Result |
+-------+-------+--------+
| Arun  |    77 | PASS   |
| Priya |    35 | FAIL   |
+-------+-------+--------+
2 rows in set
```

Video 7 · Triggers (INSERT / UPDATE / DELETE)

Step 1: main table + log table

```
mysql> CREATE DATABASE trig;
```

```
Query OK, 0 rows affected
```

```
mysql> USE trig;
```

```
Database changed
```

```
mysql> CREATE TABLE Account (
    ->     AccountNo INT PRIMARY KEY,
    ->     Name      VARCHAR(50),
    ->     Balance   DECIMAL(10,2)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Account_Log (
    ->     LogID      INT AUTO_INCREMENT PRIMARY KEY,
    ->     AccountNo  INT,
    ->     Action     VARCHAR(20),
    ->     OldBalance DECIMAL(10,2),
    ->     NewBalance DECIMAL(10,2),
    ->     ActionTime DATETIME
    -> );
```

```
Query OK, 0 rows affected
```

AFTER INSERT trigger + test

database: trig

```
mysql> DELIMITER //
```

```
mysql> CREATE TRIGGER trg_after_insert AFTER INSERT ON
Account FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    ->     VALUES (NEW.AccountNo, 'INSERT', NULL,
```

```
NEW.Balance, NOW());
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> INSERT INTO Account VALUES (1001, 'Arun', 5000);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Account VALUES (1002, 'Priya', 8000);
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Account_Log;
```

```
+-------+-----------+--------+------------+------------+---------------------+
| LogID | AccountNo | Action | OldBalance | NewBalance | ActionTime          |
+-------+-----------+--------+------------+------------+---------------------+
|     1 |      1001 | INSERT | NULL       |    5000.00 | 2026-10-15 02:59:39 |
|     2 |      1002 | INSERT | NULL       |    8000.00 | 2026-10-15 02:59:39 |
+-------+-----------+--------+------------+------------+---------------------+
2 rows in set
```

> We never inserted into Account_Log ourselves; the trigger did it.

AFTER UPDATE trigger + test

database: trig

```
mysql> DELIMITER //
```

```
mysql> CREATE TRIGGER trg_after_update AFTER UPDATE ON
Account FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    ->     VALUES (NEW.AccountNo, 'UPDATE', OLD.Balance,
NEW.Balance, NOW());
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> UPDATE Account SET Balance = 6000 WHERE AccountNo =
```

```
1001;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Account_Log;
```

```
+-------+-----------+--------+------------+------------+---------------------+
| LogID | AccountNo | Action | OldBalance | NewBalance | ActionTime          |
+-------+-----------+--------+------------+------------+---------------------+
|     1 |      1001 | INSERT | NULL       |    5000.00 | 2026-10-15 02:59:39 |
|     2 |      1002 | INSERT | NULL       |    8000.00 | 2026-10-15 02:59:39 |
|     3 |      1001 | UPDATE |    5000.00 |    6000.00 | 2026-10-15 02:59:39 |
+-------+-----------+--------+------------+------------+---------------------+
3 rows in set
```

Row 3: OLD.Balance = 5000, NEW.Balance = 6000.

AFTER DELETE trigger + test

database: trig

```
mysql> DELIMITER //
```

```
mysql> CREATE TRIGGER trg_after_delete AFTER DELETE ON
Account FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    ->     VALUES (OLD.AccountNo, 'DELETE', OLD.Balance,
NULL, NOW());
    -> END //
```

```
Query OK, 0 rows affected
```

```
mysql> DELIMITER ;
```

```
mysql> DELETE FROM Account WHERE AccountNo = 1002;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Account;
```

```
+-----------+------+---------+
| AccountNo | Name | Balance |
+-----------+------+---------+
|      1001 | Arun | 6000.00 |
+-----------+------+---------+
1 row in set
```

```
mysql> SELECT * FROM Account_Log;
```

```
+-------+-----------+--------+------------+------------+---------------------+
| LogID | AccountNo | Action | OldBalance | NewBalance | ActionTime          |
+-------+-----------+--------+------------+------------+---------------------+
|     1 |      1001 | INSERT | NULL       |    5000.00 | 2026-10-15 02:59:39 |
|     2 |      1002 | INSERT | NULL       |    8000.00 | 2026-10-15 02:59:39 |
|     3 |      1001 | UPDATE |    5000.00 |    6000.00 | 2026-10-15 02:59:39 |
|     4 |      1002 | DELETE |    8000.00 | NULL       | 2026-10-15 02:59:39 |
+-------+-----------+--------+------------+------------+---------------------+
4 rows in set
```

SHOW TRIGGERS

database: trig

```
mysql> SHOW TRIGGERS;
```

```
+------------------+--------+---------+--------+
| Trigger          | Event  | Table   | Timing |
+------------------+--------+---------+--------+
| trg_after_insert | INSERT | Account | AFTER  |
| trg_after_update | UPDATE | Account | AFTER  |
| trg_after_delete | DELETE | Account | AFTER  |
+------------------+--------+---------+--------+
3 rows in set
```

> Only the main columns are shown.

Video 8 · XML + XSD validation

XML is checked with a program, not at the mysql> prompt. These are the real outputs of **Validate.java** (see the Full Solutions PDF for the three files).

Valid XML

```
$ javac Validate.java
$ java Validate
```

```
XML is VALID according to the XSD
```

Broken XML (deleted Priya's <name> tag)

```
$ java Validate
```

```
XML is INVALID: cvc-complex-type.2.4.a: Invalid content was
found starting with element 'dept'. One of '{name}' is
expected.
```

The validator says exactly what is wrong: it expected <name> but found <dept>. Showing both the valid and the invalid run proves the XSD really checks the XML.

Video 9 · The unseen Library question (from Part 9)

Set up the new schema with sample data

```
mysql> CREATE DATABASE lib;
```

```
Query OK, 0 rows affected
```

```
mysql> USE lib;
```

```
Database changed
```

```
mysql> CREATE TABLE Library (BookID INT PRIMARY KEY, Title
VARCHAR(50), Author VARCHAR(50), Price DECIMAL(8,2));
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Member (MemberID INT PRIMARY KEY, Name
VARCHAR(50));
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Library VALUES (10,'Wings of
Fire','Kalam',300), (11,'Ponniyin Selvan','Kalki',500),
    ->
(12,'Godan','Premchand',250);
```

```
Query OK, 3 rows affected
```

```
mysql> INSERT INTO Member VALUES (1,'Arun'), (2,'Priya');
```

```
Query OK, 2 rows affected
```

Create Issue with foreign keys

database: lib

```
mysql> CREATE TABLE Issue (
    ->     IssueID   INT PRIMARY KEY,
    ->     MemberID  INT,
    ->     BookID    INT,
    ->     IssueDate DATE,
    ->     FOREIGN KEY (MemberID) REFERENCES
```

```
Member(MemberID),
    ->     FOREIGN KEY (BookID)   REFERENCES Library(BookID)
    -> );
```

```
Query OK, 0 rows affected
```

Books priced above average

database: lib

```
mysql> SELECT * FROM Library WHERE Price > (SELECT AVG(Price)
FROM Library);
```

```
+--------+-----------------+--------+--------+
| BookID | Title           | Author | Price  |
+--------+-----------------+--------+--------+
|     11 | Ponniyin Selvan | Kalki  | 500.00 |
+--------+-----------------+--------+--------+
1 row in set
```

> Average = (300 + 500 + 250) / 3 = 350.

Issue, savepoint, issue, roll back the second

database: lib

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Issue VALUES (1, 1, 10, '2025-03-01');
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Issue VALUES (2, 1, 11, '2025-03-02');
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Issue;
```

```
+---------+----------+--------+------------+
| IssueID | MemberID | BookID | IssueDate  |
+---------+----------+--------+------------+
|       1 |        1 |     10 | 2025-03-01 |
+---------+----------+--------+------------+
1 row in set
```

Count issues per member

database: lib

```
mysql> INSERT INTO Issue VALUES (3, 2, 12, '2025-03-03'), (4,
2, 11, '2025-03-04');
```

```
Query OK, 2 rows affected
```

```
mysql> SELECT MemberID, COUNT(*) AS TotalIssues FROM Issue
GROUP BY MemberID;
```

```
+----------+-------------+
| MemberID | TotalIssues |
+----------+-------------+
|        1 |           1 |
|        2 |           2 |
+----------+-------------+
2 rows in set
```